Polytech Marseille – Informatique, 4e année

Module : Analyse d'image

Détection de contours

Comparaison des détecteurs de Sobel, Laplace et Canny





GUESSOUM Ahyat

Enseignant : Sébastien Mavromatis

24 octobre 2026

1 Introduction

L'objectif de ce TP est de comparer les trois détecteurs de contours vus en cours : Sobel, Laplace et Canny. Chaque détecteur à des paramètres à régler (le flou et un ou deux seuils). Pour comparer les détecteurs avec des chiffres, on utilise la base d'images de l'université de Groningen [2]. Elle contient 40 photos en niveaux de gris, et pour chaque photo une image de référence où les contours ont été tracés à la main. En comparant les contours obtenus avec ceux de référence, on peut calculer la performance, le taux de faux positifs et le taux de faux négatifs de chaque détecteur.

Le programme est écrit en C++ avec OpenCV. Il permet de choisir le détecteur, de régler ses paramètres avec des curseurs et de lancer le calcul sur les 40 images.

2 Théorie

2.1 Contours et dérivées

Une image en niveaux de gris est un tableau de valeurs entre 0 (noir) et 255 (blanc). Un contour est un endroit où l'intensité change brusquement, par exemple au bord d'un objet. Sur une ligne de pixels,on a par exemple :

$$10 \quad 10 \quad 10 \quad 200 \quad 200 \quad 200$$

Ici le contour est entre le 3e et le 4e pixel. Pour trouver les contours, on cherche donc les endroits où l'image varie beaucoup, c'est-à-dire là où la dérivée est grande.

Avant de dériver, on applique un flou gaussien. Sur une vraie photo il y a du bruit et des textures (herbe, fourrure…) qui créent plein de petites variations, et donc de faux contours. Le flou les atténue. Sa force est donnée par l'écart-type σ : si σ est trop petit le bruit reste, et s'il est trop grand les vrais contours deviennent flous.

2.2 Sobel

Sobel calcule la dérivée/gradient en x et en y avec deux masques 3×3 :

$$G_x : \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \qquad G_y : \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

G_x détecte les contours verticaux et G_y les contours horizontaux. On les combine avec la norme :

$$\|\nabla I\| = \sqrt{G_x^2 + G_y^2}$$

Un pixel est un contour si cette norme est plus grande qu'un seuil. C'est simple, mais les traits obtenus sont souvent épais, et avec un seul seuil il est difficile d'enlever le bruit sans perdre des contours.

2.3 Laplace

Le Laplacien utilise la dérivée seconde :

$$\Delta I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2} \qquad \text{avec par exemple le masque} \quad \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Avec l'exemple $10, 10, 10, 200, 200, 200$, la dérivée première donne $0, 0, 190, 0, 0$ et la dérivée seconde $0, 190, -190, 0$. Au niveau du contour, la dérivée seconde passe du positif au négatif : c'est un passage par zéro. Avec Laplace, on cherche donc les passages par zéro.

Le problème est que la dérivée seconde est très sensible au bruit, donc il faut un flou plus fort, et ne garder que les passages par zéro assez « forts ».

2.4 Canny

Canny part aussi du gradient, mais il ajoute des étapes. Après le flou et le calcul du gradient, il amincit les contours pour n'avoir que des traits d'un pixel (suppression des non-maxima). Ensuite il utilise deux seuils (hystérésis) : un pixel au-dessus du seuil haut est gardé, un pixel en dessous du seuil bas est enlevé, et un pixel entre les deux est gardé seulement s'il est relié à un contour déjà gardé. Ça permet d'avoir des contours plus continus et moins de bruit.

3 Programme

3.1 Organisation

Le programme est en C++ avec OpenCV 4 et se compile avec CMake. J'ai réutilisé deux classes du livre utilisé dans le cours [1] :

— `edgedetector.h` (chapitre 7) : classe `EdgeDetector` pour Sobel ;
— `laplacianZC.h` (chapitre 6) : classe `LaplacianZC` pour le Laplacien et les passages par zéro.

Le reste est dans trois fichiers : `detecteurs.h` (les trois détecteurs), `evaluation.h` (le calcul des mesures) et `main.cpp` (l'interface et le traitement des 40 images). Chaque détecteur renvoie une image en noir et blanc, avec les contours en blanc.

3.2 Fonctions OpenCV utilisées

Les trois détecteurs commencent par un flou gaussien, où `sigma` est le σ de la gaussienne :

```
cv::GaussianBlur(image, lisse, cv::Size(0, 0), sigma);
```

Sobel. La classe `EdgeDetector` calcule G_x et G_y avec `cv::Sobel` puis la norme. Ensuite je seuille la norme avec `cv::threshold` :

```
EdgeDetector ed;
ed.setAperture(3);
ed.computeSobel(lisse);
cv::threshold(ed.getMagnitude(), contours, seuil, 255, cv::THRESH_BINARY);
```

Laplace. La classe `LaplacianZC` calcule le Laplacien (`cv::Laplacian`) et trouve les passages par zéro. Pour garder seulement les passages forts, j'utilise le gradient morphologique du chapitre 5, qui donne l'écart entre la valeur la plus grande et la plus petite autour du pixel :

```
cv::Mat lap = laplacien.computeLaplacian(lisse);
cv::Mat passages = laplacien.getZeroCrossings(lap);
cv::morphologyEx(lap, pente, cv::MORPH_GRADIENT, cv::Mat());
cv::threshold(pente, fort, seuil, 255, cv::THRESH_BINARY);
contours = passages & fort;   // passage par zero ET quil soit assez fort
```

Canny. OpenCV fait directement le gradient, l'amincissement et l'hystérésis :

```
cv::Canny(lisse, contours, seuilBas, seuilHaut, 3, true);
```

`seuilBas` et `seuilHaut` sont les deux seuils, `3` la taille du masque de Sobel, et `true` demande la même norme que pour Sobel. Comme `cv::Canny` ne fait pas le flou, je l'applique avant.

Curseur	Fonction	Rôle
Flou (σ)	`GaussianBlur`	force du flou
Seuil	`threshold`	seuil de Sobel / Laplace, seuil bas de Canny
Seuil haut	`Canny`	seuil haut de l'hystérésis

TABLE 1 – Paramètres réglables et fonctions associées.

3.3 Interface

Au lancement, le programme charge les 40 images et leurs références (`cv::glob`). La fenêtre (figure 1) a six curseurs (`cv::createTrackbar`) : image, détecteur, flou, seuil, seuil haut et voisinage. Quand on bouge un curseur, le programme refait la détection et affiche l'image, la référence, les contours trouvés et une image des erreurs, avec les mesures en haut.

Il y a aussi trois touches : `l` lance le calcul sur les 40 images avec les réglages des curseurs, `a` lance une série de réglages enregistrés dans le code (3 par détecteur, avec et sans voisinage) et écrit les moyennes dans un fichier CSV, et `s` enregistre une capture.

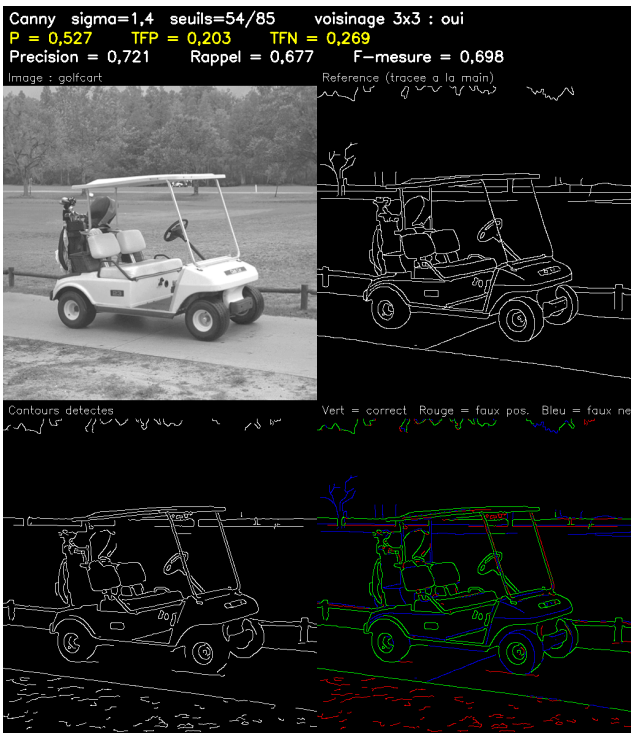


FIGURE 1 – Fenêtre du programme (Canny sur *golfcart*). En bas à droite : vert = correct, rouge = faux positif, bleu = faux négatif.

4 Évaluation

4.1 Mesures

Pour chaque image, on compte le nombre de pixels de contour détectés, le nombre de pixels de contour dans la référence, les pixels corrects (dans les deux), les faux positifs FP (détectés mais pas dans la référence) et les faux négatifs FN (dans la référence mais pas détectés). Avec T =

corrects $+ FP + FN$, on calcule :

$$P = \frac{\text{corrects}}{T} \qquad TFP = \frac{FP}{T} \qquad TFN = \frac{FN}{T}$$

Pour compter les pixels, on utilise `cv::countNonZero`, et `&` pour garder les pixels blancs dans deux images à la fois.

4.2 Voisinage 3×3

Les contours tracés à la main ne sont pas précis au pixel près. Sans tolérance, un contour décalé d'un pixel compte comme une erreur. Comme proposé dans le sujet, on accepte donc un pixel s'il y a un pixel correspondant juste à côté. Pour ça, on « grossis » les contours d'un pixel avec une dilatation (chapitre 5) :

```
cv::Mat element(3, 3, CV_8U, cv::Scalar(1));
cv::dilate(reference, zoneReference, element);
cv::dilate(detecte, zoneDetecte, element);
corrects = cv::countNonZero(detecte & zoneReference);
FP = detectes - corrects;
FN = reference_total - cv::countNonZero(reference & zoneDetecte);
```

Pour FN, je n'ai pas utilisé directement référence − corrects. Avec la tolérance, un trait épais peut donner plusieurs pixels corrects pour un seul pixel de référence, et FN pouvait devenir négatif (ça arrivait souvent avec Sobel). Je compte donc les pixels de référence qui n'ont aucun pixel détecté à côté. Sans voisinage, ça revient à la formule du sujet.

4.3 Autres critères

J'ai aussi calculé la précision, le rappel et la F-mesure :

$$\text{Précision} = \frac{\text{corrects}}{\text{détectés}} \qquad \text{Rappel} = \frac{\text{référence trouvés}}{\text{référence}} \qquad F = \frac{2 \times \text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

La précision indique quelle part de ce qu'on détecté est juste, et le rappel indique la part des contours de la référence retrouvée. Il ne faut pas regarder la précision seule : avec un seuil très haut on détecte très peu de contours, donc la précision est bonne alors que le résultat à l'image est mauvais.

Le programme affiche aussi une image des erreurs en couleur (avec `cv::merge`, chapitre 3). Elle m'a beaucoup aidée pour régler les seuils : s'il y a trop de rouge il faut monter le seuil, s'il y a trop de bleu il faut le baisser.

5 Résultats

J'ai d'abord cherché un bon réglage pour chaque détecteur avec les curseurs, en testant plusieurs images avec le voisinage activé. J'ai gardé : Sobel ($\sigma = 1{,}4$, seuil 92), Laplace ($\sigma = 1{,}9$, seuil 21) et Canny ($\sigma = 1{,}4$, seuils 54/85). Pour voir l'effet du seuil, j'ai aussi testé un seuil plus faible et un seuil plus fort pour chaque détecteur. Les tableaux donnent les moyennes sur les 40 images.

Détecteur	Réglage	P	TFP	TFN	Précision	Rappel	F
Sobel (σ=1,4)	seuil 55	0,328	0,620	0,050	0,351	0,772	0,456
	seuil 92	**0,406**	0,351	0,242	0,556	0,498	**0,492**
	seuil 138	0,302	0,131	0,565	0,732	0,257	0,357
Laplace (σ=1,9)	seuil 13	0,158	0,788	0,052	0,170	0,765	0,265
	seuil 21	**0,223**	0,536	0,240	0,317	0,534	**0,362**
	seuil 32	0,175	0,230	0,594	0,501	0,274	0,318
Canny (σ=1,4)	seuils 32/51	0,228	0,699	0,071	0,252	0,784	0,359
	seuils 54/85	**0,317**	0,455	0,227	0,438	0,616	**0,473**
	seuils 81/128	0,297	0,208	0,493	0,640	0,403	0,459

TABLE 2 – Moyennes sur les 40 images, avec voisinage 3×3.

Détecteur	Réglage	P	TFP	TFN	Précision	Rappel	F
Sobel	seuil 55	0,112	0,812	0,075	0,124	0,632	0,199
	seuil 92	**0,148**	0,573	0,277	0,215	0,378	**0,256**
	seuil 138	0,120	0,292	0,586	0,316	0,186	0,212
Laplace	seuil 13	0,056	0,814	0,129	0,067	0,312	0,105
	seuil 21	**0,081**	0,584	0,333	0,137	0,208	**0,148**
	seuil 32	0,070	0,285	0,643	0,240	0,103	0,129
Canny	seuils 32/51	0,092	0,734	0,173	0,117	0,353	0,165
	seuils 54/85	**0,132**	0,530	0,337	0,219	0,289	0,229
	seuils 81/128	**0,132**	0,302	0,564	0,338	0,196	**0,230**

TABLE 3 – Moyennes sur les 40 images, sans voisinage.

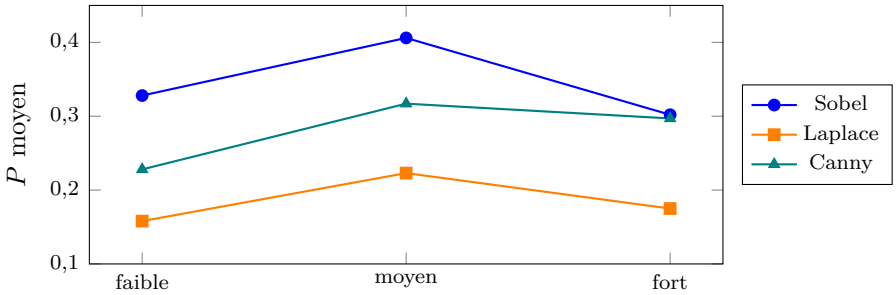


FIGURE 2 – P moyen selon le seuil (avec voisinage).



(a) Sobel : $P = 0{,}620$ (b) Laplace : $P = 0{,}458$ (c) Canny : $P = 0{,}527$

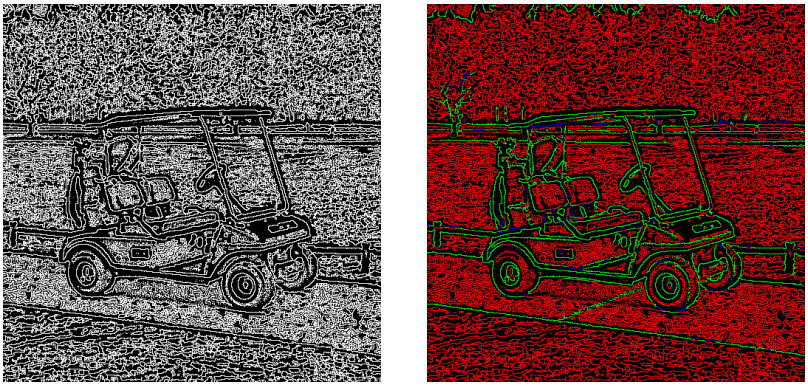
FIGURE 3 – Erreurs sur *golfcart* avec le meilleur réglage de chaque détecteur.

6 Interprétation

6.1 Effet du seuil

Pour les trois détecteurs, le meilleur P est obtenu avec le seuil moyen (figure 2). Avec un seuil faible, il y a beaucoup de faux positifs (TFP entre 0,62 et 0,79), et avec un seuil fort beaucoup de contours sont oubliés (TFN entre 0,49 et 0,59). Il faut donc trouver un compromis. On le voit aussi avec la précision et le rappel : quand le seuil augmente, la précision augmente mais le rappel diminue.

Avec le Laplacien et un seuil de 3 (figure 4), presque toute l'image est détectée comme contour : la précision tombe à 0,112. Ça montre bien que la dérivée seconde est très sensible au bruit.



(a) Contours détectés (b) Erreurs

FIGURE 4 – Laplace avec un seuil trop bas (3).

6.2 Comparaison des détecteurs

Avec voisinage, Sobel a le meilleur P (0,406), puis Canny (0,317) et Laplace (0,223). Laplace est le moins bon, ce qui est logique vu sa sensibilité au bruit : il garde beaucoup de faux positifs dans les zones texturées, comme le sol de *golfcart* (figure 3).

Je m'attendais normalement à ce que Canny soit le meilleur, et visuellement ses contours sont les plus propres (fins et continus). Je pense que Sobel est avantagé par le voisinage : ses traits sont épais, donc avec la tolérance beaucoup de ses pixels comptent comme corrects, alors qu'un trait fin de Canny ne compte qu'une fois. Sans voisinage, l'écart entre les deux est beaucoup plus petit (0,148 contre 0,132). Mon réglage de Canny n'est peut-être pas le meilleur non plus, car je l'ai choisi en testant seulement quelques images.

Sans voisinage, tous les P sont beaucoup plus bas (environ 2,5 fois). Donc beaucoup de contours sont trouvés à un pixel près, mais pas exactement au même endroit que le tracé à la main. Le voisinage est donc utile pour faire une comparaison plus juste.

6.3 Selon les images

Les résultats changent beaucoup d'une image à l'autre. Les meilleures sont *golfcart*, *hyena* et *elephant*, avec des objets bien contrastés sur un fond assez uni. Les moins bonnes sont *bear_7*, *bear_8* et *bear_6*, à cause de la fourrure et du décor : sur *bear_7*, la plupart des pixels détectés par Sobel sont des faux positifs. Un seul réglage ne peut donc pas convenir à toutes les images.

6.4 Limites

— Les images de référence ne contiennent que les contours des objets, pas ceux des textures (herbe, poils…). Ces contours comptent donc comme des faux positifs, ce qui explique en

partie pourquoi les P sont assez bas pour tous les détecteurs.
— Le voisinage avantage les détecteurs qui donnent des traits épais, comme Sobel.
— J'ai utilisé un seul réglage pour les 40 images, choisi à la main sur quelques images seulement.

Pour améliorer, on pourrait chercher les seuils automatiquement sur toutes les images, ou amincir les traits de Sobel avant de les comparer.

7 Conclusion

Dans ce TP, j'ai programmé et comparé trois détecteurs de contours. Mon programme permet de choisir le détecteur, de régler les paramètres avec des curseurs, de traiter les 40 images et de calculer les mesures demandées, avec ou sans voisinage, ainsi que la précision, le rappel et la F-mesure.

Avec mes réglages, Sobel obtient le meilleur résultat, devant Canny puis Laplace. Mais Sobel est en partie avantagé par la façon dont on compare les images, et c'est Canny qui donne les contours les plus propres à l'œil. Ce TP montre qu'il faut faire attention à ce que mesurent vraiment les chiffres.

Références

[1] R. Laganière, *OpenCV 3 Computer Vision Application Programming Cookbook*, 3e édition, Packt Publishing, 2016 (code des chapitres 3, 5, 6 et 7).

[2] Base de contours de l'université de Groningen : `http://www.cs.rug.nl/~imaging/databases/contour_database/`

[3] Documentation OpenCV 4 : `https://docs.opencv.org/4.x/`

[4] ClaudeIA : aide à la correction des erreurs de compilation (main.cpp) et à l'analyse des résultats (720 lignes de mesures). Le réglage des détecteurs, les mesures et les captures ont été faits avec mon programme.